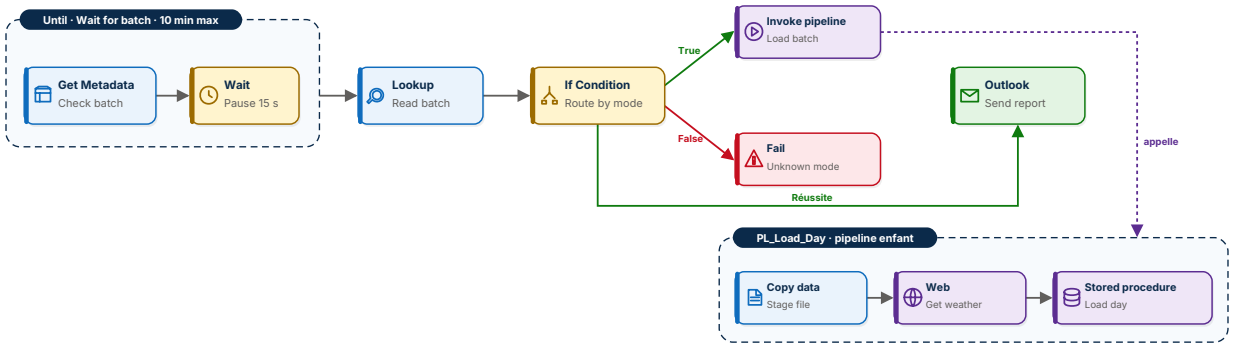
MICROSOFT FABRIC · DATA FACTORY · GUIDE DE RÉFÉRENCE

Pipeline Microsoft Fabric : attendre, router, appeler, remplacer

ATTENDRE ROUTER APPELER REMPLACER

Un pipeline chef d'orchestre attend l'arrivée d'un lot, lit son manifeste, choisit la route selon le mode du lot, appelle un pipeline enfant qui charge les ventes, interroge la météo du jour sur une API publique et remplace la journée dans l'entrepôt par une procédure stockée. Deux pipelines, onze activités, onze types, aucune ligne de code.



PL_Orchestrator appelle PL_Load_Day. Jaune : le contrôle (attendre, router). Violet : les appels (pipeline, API, procédure). Bleu : les données. Rouge : le refus d'un lot inconnu.

2	11	11	5	7
pipelines	activités	types d'activité	types nouveaux	pages

AUTEUR
Abdoul Hamid Diallo, Microsoft Data & AI Engineer

PRÉREQUIS
Un espace de travail Fabric sur une capacité, une boîte Outlook Microsoft 365, l'accès à api.open-meteo.com

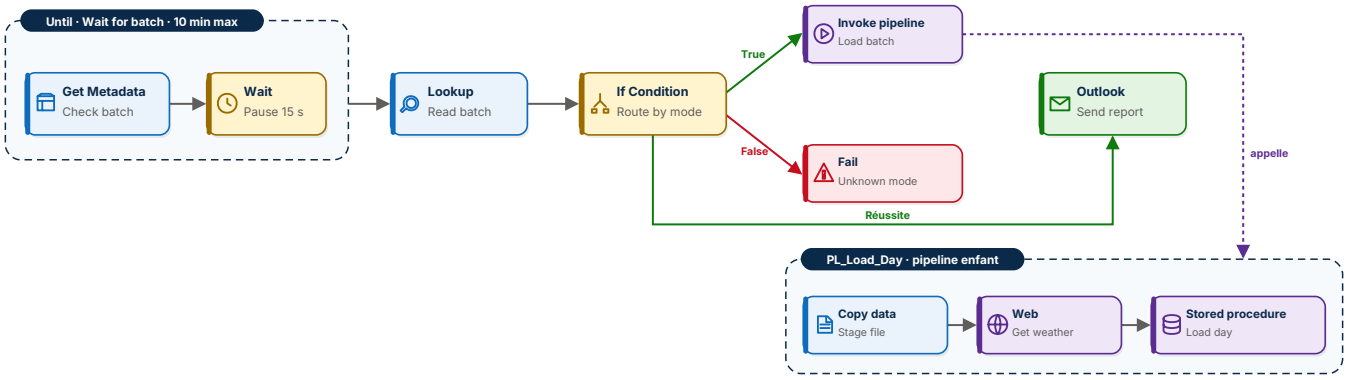
FICHIERS DU PROJET
Le dossier data fourni avec ce guide : six fichiers de ventes, six manifestes et expressions.txt

ÉDITION
Octobre 2026

VUE D'ENSEMBLE

Ce que vous construisez

Trois cafés, Metz, Nancy et Thionville, déposent chaque soir leurs lignes de vente. Quand le lot est complet, un petit manifeste JSON arrive à son tour : il dit quel fichier charger, pour quelle journée, et dans quel mode. Le pipeline chef d'orchestre ne fait rien tant que ce manifeste n'est pas là. Puis il route, délègue, enrichit, remplace. C'est le schéma d'un pipeline de production : un orchestrateur qui décide, un enfant qui travaille. Cinq types d'activité nouveaux : Until, Wait, Invoke pipeline, Web, Stored procedure.



Deux pipelines. En haut, l'orchestrateur : une boucle Until qui attend, un Lookup qui lit, une Condition Si qui appelle l'enfant ou refuse le lot, un e-mail. En bas, l'enfant appelé : Copy vers la zone de transit, Web vers l'API météo, procédure stockée qui remplace la journée. Les libellés **en bleu** sont les mots exacts de l'interface Fabric, les blocs bleu clair se copient tels quels.

Les onze activités

ACTIVITÉ	TYPE	RÔLE
`Wait for batch`	Until	Boucle jusqu'à ce que le manifeste du lot existe, dix minutes au plus
`Check batch`	Get Metadata	Dans la boucle : le fichier `batch_<lot>.json` existe-t-il ?
`Pause`	Wait	Dans la boucle : quinze secondes avant de revérifier
`Read batch`	Lookup	Lit le manifeste : `mode`, `file`, `run_date`, `batch_id`
`Route by mode`	If Condition	Le mode est-il `standard` ou `correction` ?
`Load batch`	Invoke pipeline	Branche True : appelle `PL_Load_Day` avec les quatre valeurs du manifeste
`Unknown mode`	Fail	Branche False : refuse le lot et nomme le mode reçu
`Send report`	Office 365 Outlook	Le bilan du lot, après la condition
`Stage file`	Copy data, enfant	Copie le fichier de ventes dans la table de transit de l'entrepôt
`Get weather`	Web, enfant	Interroge l'API Open-Meteo : température maximale et pluie du jour à Metz
`Load day`	Stored procedure, enfant	Remplace la journée dans `fact_sales`, journalise les corrections, vide le transit

Le jeu de données et les lots

LOT	MODE	FICHIER
`2026-10-04-1`	standard	`sales_2026-10-04.csv`, 226 lignes
`2026-10-05-1`	standard	`sales_2026-10-05.csv`, 230 lignes
`2026-10-06-1`	standard	`sales_2026-10-06.csv`, 257 lignes, 9 salades en double à Nancy
`2026-10-06-2`	**correction**	`sales_2026-10-06_corrige.csv`, 248 lignes
`2026-10-07-1`	**test**	mode inconnu : le lot doit être refusé
`2026-10-08-1`	standard	`sales_2026-10-08.csv`, déposé pendant l'attente

Un fichier de ventes par jour, une ligne par article vendu : `sale_date`, `sale_time`, `shop`, `product`, `qty`, `unit_price`. Dix produits, trois cafés, environ 240 lignes par jour ; `fact_sales` en garde 30 par journée, une par café et produit, avec la météo. Chaque lot a son manifeste `batch_<lot>.json`, quatre clés : `{ "batch_id": "2026-10-06-2", "mode": "correction", "file": "sales_2026-10-06_corrige.csv", "run_date": "2026-10-06" }`. Six lots, sept exécutions : un refus puis une reprise, et un lot déposé pendant que le pipeline l'attend.

ÉTAPE 1

Préparer, puis construire le pipeline enfant

1 · ENFANT | 2 · ATTENDRE ET LIRE | 3 · ROUTER ET APPELER | 4 · EXÉCUTER

1 Les objets Fabric

1. Lakehouse **LH_Market**. Sur **Fichiers**, **…**, **Nouveau sous-dossier** `inbox` ; ouvrez-le, puis **Charger des fichiers** : **tous les CSV et JSON de `data` sauf `batch_2026-10-08-1.json`**, gardé pour la dernière exécution. Ils doivent être dans `inbox`, pas à la racine de Fichiers.
2. Entrepôt **WH_Market**, **Nouvelle requête SQL**, exécutez ce script en entier :

```
CREATE TABLE dbo.stg_sales (sale_date VARCHAR(10), sale_time VARCHAR(16), shop VARCHAR(20), product VARCHAR(40), qty VARCHAR(10),
unit_price VARCHAR(10));
CREATE TABLE dbo.fact_sales (sale_date DATE NOT NULL, shop VARCHAR(20) NOT NULL, product VARCHAR(40) NOT NULL, qty INT NOT NULL, revenue
DECIMAL(10,2) NOT NULL, temp_max_c DECIMAL(5,1), rain_mm DECIMAL(6,1), batch_id VARCHAR(20) NOT NULL, loaded_at DATETIME2(6) NOT NULL);
CREATE TABLE dbo.audit_restatements (run_date DATE NOT NULL, shop VARCHAR(20) NOT NULL, batch_id VARCHAR(20) NOT NULL, revenue_before
DECIMAL(10,2), revenue_after DECIMAL(10,2), restated_at DATETIME2(6) NOT NULL);
GO
CREATE PROCEDURE dbo.usp_load_day @run_date DATE, @batch_id VARCHAR(20), @mode VARCHAR(20), @temp_max DECIMAL(5,1), @rain_mm DECIMAL(6,1)
AS
BEGIN
  IF @mode = 'correction'
    INSERT INTO dbo.audit_restatements SELECT @run_date, b.shop, @batch_id, b.rev, a.rev, GETDATE()
    FROM (SELECT shop, SUM(revenue) AS rev FROM dbo.fact_sales WHERE sale_date = @run_date GROUP BY shop) b
    JOIN (SELECT shop, SUM(CAST(qty AS INT) * CAST(unit_price AS DECIMAL(10,2))) AS rev FROM dbo.stg_sales GROUP BY shop) a ON a.shop =
b.shop;
  DELETE FROM dbo.fact_sales WHERE sale_date = @run_date;
  INSERT INTO dbo.fact_sales SELECT @run_date, shop, product, SUM(CAST(qty AS INT)), SUM(CAST(qty AS INT) * CAST(unit_price AS
DECIMAL(10,2))), @temp_max, @rain_mm, @batch_id, GETDATE()
  FROM dbo.stg_sales GROUP BY shop, product;
  TRUNCATE TABLE dbo.stg_sales;
END
```

LA TRANSFORMATION TIENT DANS LA PROCÉDURE Agréger par café et produit, remplacer la journée entière au lieu d'ajouter, noter avant et après en mode correction : rejouer un lot donne toujours le même résultat.

2 PL_Load_Day, le pipeline enfant

1. Créez le pipeline **PL_Load_Day**. Onglet **Paramètres**, quatre paramètres de type `String`, sans valeur par défaut : `batch_id`, `mode`, `file`, `run_date`.

3 Stage file

COPY DATA

1. **Copier les données**, nom `Stage file`. **Source** : **Lakehouse**, `LH_Market`, **Fichiers**, **DelimitedText**, en-tête coché, dossier `inbox`, nom de fichier :

```
@{pipeline().parameters.file}
```

2. **Destination** : **Entrepôt**, `WH_Market`, table `dbo.stg_sales`, action **Ajouter**. Laissez la mise en lots (staging) activée : l'entrepôt l'exige.

4 Get weather

WEB

1. Ajoutez **Web**, nom `Get weather`, relié à la sortie verte du Copy. **Connexion**, **Plus**, type **Web v2** : URL de base `https://api.open-meteo.com`, authentification **Anonyme**, nom `open_meteo`.
2. **Méthode** GET, **URL relative** en contenu dynamique : le bloc sous les deux colonnes.

5 Load day

STORED PROCEDURE

1. Ajoutez **Procédure stockée**, nom `Load day`, reliée à la sortie verte de `Get weather`. Connexion **Entrepôt**, `WH_Market`, procédure `[dbo].[usp_load_day]`, puis **Importer** les paramètres et donnez leurs valeurs :

run_date	`@pipeline().parameters.run_date`
batch_id	`@pipeline().parameters.batch_id`
mode	`@pipeline().parameters.mode`
temp_max	`@activity('Get weather').output.daily.temperature_2m_max[0]`
rain_mm	`@activity('Get weather').output.daily.precipitation_sum[0]`

2. Types importés : `run_date` DateTime, `batch_id` et `mode` String, `temp_max` et `rain_mm` Decimal. **Enregistrer** : le point à côté du nom de l'onglet doit disparaître, sinon l'orchestrateur appellera l'ancienne version.

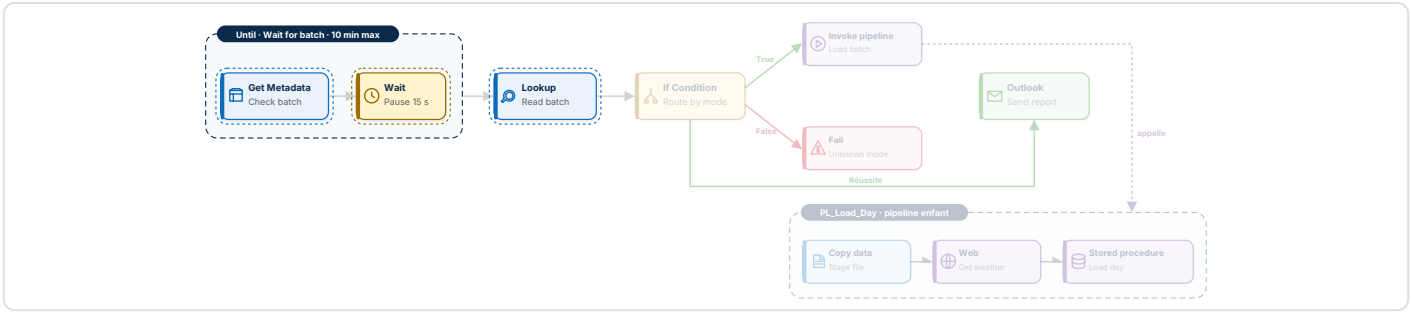
URL relative de `Get weather` (réponse : `output.daily.temperature_2m_max[0]`, `output.daily.precipitation_sum[0]`, valeurs réelles de Metz) :

```
/v1/forecast?
latitude=49.12&longitude=6.18&daily=temperature_2m_max,precipitation_sum&timezone=Europe%2FParis&start_date=@{pipe
line().parameters.run_date}&end_date=@{pipeline().parameters.run_date}
```

ÉTAPE 2

L'orchestrateur : attendre le lot, lire le manifeste

1 · ENFANT | 2 · ATTENDRE ET LIRE | 3 · ROUTER ET APPELER | 4 · EXÉCUTER



6 PL_Orchestrator et son paramètre

1. Créez le pipeline **PL_Orchestrator**. Onglet **Paramètres** :

Nom	`batch_id`
Type	`String`
Valeur par défaut	`2026-10-04-1`

UN SEUL PARAMÈTRE L'identifiant du lot suffit : tout le reste, la date, le fichier, le mode, sera lu dans le manifeste. L'orchestrateur ne connaît ni les noms de fichiers ni les dates.

7 Wait for batch — UNTIL

1. Ajoutez **Jusqu'à** (Until), nom `Wait for batch`. **Paramètres**, **Expression** :

```
@activity('Check batch').output.exists
```

Délai d'expiration : `0.00:10:00`.

2. Cliquez sur le crayon pour entrer dans la boucle.

UNTIL ÉVALUE À LA FIN DE CHAQUE TOUR Le corps s'exécute d'abord, l'expression est testée ensuite. Un tour = vérifier, attendre. Tant que le fichier manque, on recommence ; au bout de dix minutes sans manifeste, l'exécution est marquée en échec, ce qui est le comportement voulu.

8 Check batch — GET METADATA · DANS LA BOUCLE

1. Dans la boucle, **Obtenir les métadonnées**, nom `Check batch`. **Lakehouse**, `LH_Market`, **Fichiers**, dossier `inbox`, nom de fichier en contenu dynamique :

```
batch_@{pipeline().parameters.batch_id}.json
```

Format **JSON**. **Liste des champs**, **Nouveau** : **Existe**.

9 Pause — WAIT · DANS LA BOUCLE

1. Ajoutez **Attendre**, nom `Pause`, relié à la sortie verte de `Check batch`. **Temps d'attente en secondes** : `15`.
2. Fil d'Ariane : revenez au canevas principal.

POURQUOI GET METADATA ET NON LOOKUP POUR ATTENDRE
Un Lookup sur un fichier absent échoue et casse la boucle. Get Metadata avec le champ Existe renvoie simplement `false`, sans erreur : c'est le seul test d'existence propre dans un pipeline.

10 Read batch — LOOKUP

1. Ajoutez **Rechercher**, nom `Read batch`, relié à la sortie verte de `Wait for batch`. **Lakehouse**, `LH_Market`, **Fichiers**, dossier `inbox`, même nom de fichier que ci-dessus, format **JSON**, **Première ligne uniquement** coché.

CE QUE RENVOIE L'ACTIVITÉ
`output.firstRow.mode`, `.file`, `.run_date`, `.batch_id` : les quatre clés du manifeste, prêtes pour la condition et pour l'enfant.

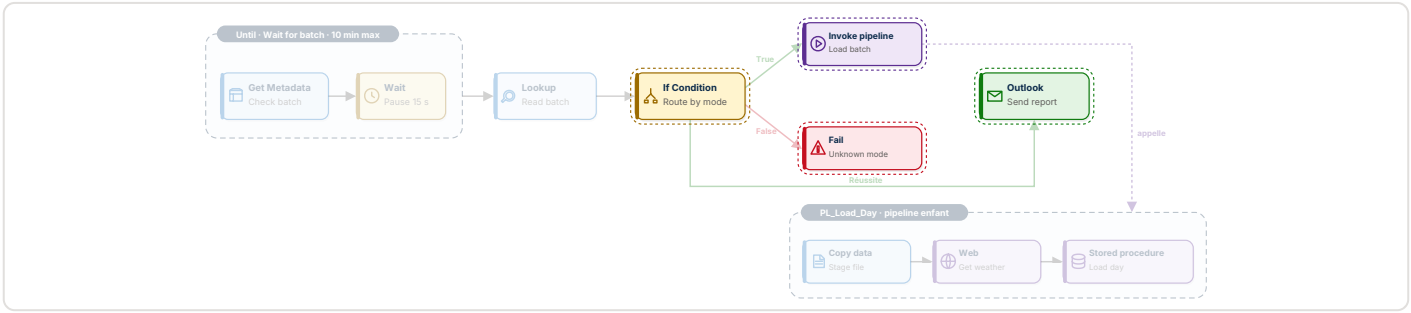
AVANT DE ROUTER

- ☐ l'expression du Until vise `Check batch`, avec le délai de dix minutes
- ☐ `Check batch` demande le champ Existe
- ☐ `Pause` est relié derrière `Check batch`, dans la boucle
- ☐ Enregistrer, le point a disparu

ÉTAPE 3

Router selon le mode, appeler l'enfant, prévenir

1 · ENFANT | 2 · ATTENDRE ET LIRE | **3 · ROUTER ET APPELER** | 4 · EXÉCUTER



11 Route by mode — IF CONDITION

1. Ajoutez une **Condition Si**, nom `Route by mode`, reliée à la sortie verte de `Read batch`. Onglet **Activités**, **Expression** :

```
@or(equals(activity('Read batch').output.firstRow.mode, 'standard'), equals(activity('Read batch').output.firstRow.mode, 'correction'))
```

UNE SEULE QUESTION SUFFIT Les deux modes connus appellent le même enfant avec les mêmes valeurs : c'est la procédure qui, en recevant `correction`, journalise avant de remplacer. La condition ne décide qu'une chose : le mode est-il connu ?

12 Load batch — INVOKE PIPELINE · TRUE

1. Dans la case **True**, ajoutez **Appeler le pipeline**, nom `Load batch`. **Paramètres** : espace de travail courant, pipeline `PL_Load_Day`, **Attendre la fin** coché. Les quatre paramètres de l'enfant apparaissent ; donnez-leur :

batch_id	`@activity('Read batch').output.firstRow.batch_id`
mode	`@activity('Read batch').output.firstRow.mode`
file	`@activity('Read batch').output.firstRow.file`
run_date	`@activity('Read batch').output.firstRow.run_date`

13 Unknown mode — FAIL · FALSE

1. Dans la case **False**, ajoutez **Échec**, nom `Unknown mode`. **Code d'erreur** `400`, **Message d'échec** :

```
@concat('Mode de lot inconnu : ', activity('Read batch').output.firstRow.mode)
```

UN LOT INCONNU NE SE CHARGE PAS Ni tenté, ni ignoré en silence : refusé, avec le mode reçu dans le message. L'e-mail de bilan, sur le fil vert, ne part pas.

14 Send report — OFFICE 365 OUTLOOK

1. Sur le canevas principal, **E-mail Office 365**, nom `Send report`, relié à la sortie verte de `Route by mode`. Connectez votre compte, renseignez **À**.
2. **Objet** :

```
Lot @{pipeline().parameters.batch_id} chargé en mode @{activity('Read batch').output.firstRow.mode}
```

3. **Corps**, au clavier, valeurs insérées par l'éclair, onglet **Fonctions** :

```
Journée : activity('Read batch').output.firstRow.run_date
Fichier : activity('Read batch').output.firstRow.file
```

4. **Valider**, **Enregistrer**.

TROIS FINS, TOUTES PROPRES Chargé en standard, chargé en correction avec journal, ou refusé avec le mode en clair. Et avant tout cela, dix minutes de patience si le lot tarde.

ÉTAPE 4

Exécuter et vérifier

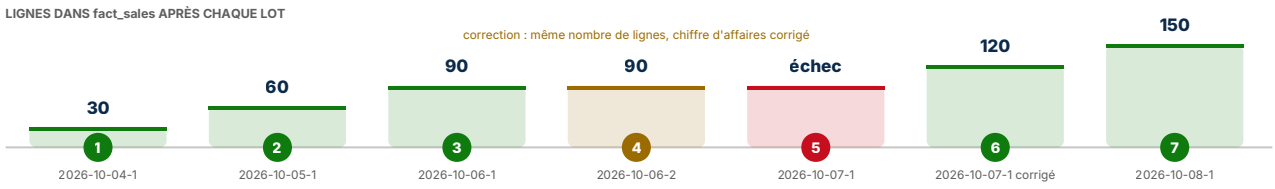
1 · ENFANT | 2 · ATTENDRE ET LIRE | 3 · ROUTER ET APPELER | **4 · EXÉCUTER**

OBJECTIF

Sept exécutions de `PL_Orchestrator` : trois lots standard, une correction, un refus, une reprise, et un lot déposé pendant l'attente.

RÉSULTAT

150 lignes dans `fact_sales`, 5 512,60 de chiffre d'affaires, 3 lignes d'audit.



Exécutions 1 à 3 — batch_id = 2026-10-04-1, puis 2026-10-05-1, puis 2026-10-06-1

Exécuter, saisissez le lot. `Wait for batch` fait un seul tour, la condition passe en True, l'enfant tourne, l'e-mail part. La table passe à **30**, **60**, puis **90** lignes ; la journée du 6 vaut 1 234,00 dont Nancy 470,10.

Exécution 4 — batch_id = 2026-10-06-2, la correction

Condition True, mode `correction`. Toujours **90** lignes, mais la journée du 6 vaut maintenant 1 157,50 et Nancy 393,60 : les 9 salades en double ont disparu. `audit_restatements` contient trois lignes, une par café, et seule celle de Nancy change : 470,10 avant, 393,60 après.

Exécution 5 — batch_id = 2026-10-07-1, mode test

`Unknown mode` lève « Mode de lot inconnu : test », exécution marquée **Échec**, pas d'e-mail, table intacte. Corrigez le manifeste sur votre poste (`test` devient `standard`) et redéposez-le dans `inbox` à la place de l'ancien.

Exécution 6 — batch_id = 2026-10-07-1, à nouveau

Condition True, **120** lignes, journée du 7 à 1 042,50.

Exécution 7 — batch_id = 2026-10-08-1, le manifeste n'est pas encore déposé

Lancez d'abord : `Check batch` renvoie `exists = false`, `Pause`, et on recommence. Déposez alors `batch_2026-10-08-1.json` dans `inbox` : au tour suivant la boucle sort, le lot se charge, **150** lignes, e-mail reçu.

Vérifier dans l'entrepôt

Ouvrez `WH_Market`, **Nouvelle requête SQL** :

```
SELECT sale_date, COUNT(*) AS lignes, SUM(qty) AS
articles,
  SUM(revenue) AS ca, MAX(temp_max_c) AS temp_max,
  MAX(rain_mm) AS pluie, MAX(batch_id) AS lot
FROM dbo.fact_sales GROUP BY sale_date ORDER BY
sale_date;

SELECT * FROM dbo.audit_restatements;
```

SALE_DATE	LIGNES	ARTICLES	CA	LOT
2026-10-04	30	309	1 076,20	2026-10-04-1
2026-10-05	30	317	1 039,00	2026-10-05-1
2026-10-06	30	343	1 157,50	**2026-10-06-2**
2026-10-07	30	315	1 042,50	2026-10-07-1
2026-10-08	30	342	1 197,40	2026-10-08-1

150
lignes, cinq journées, chacune en un seul exemplaire

5 512,60
chiffre d'affaires total après correction

76,50
retirés par la correction : 9 salades à 8,50

5 / 5
journées avec température et pluie renseignées

CE QUE CES CHIFFRES PROUVENT `temp_max` et `pluie` sont remplies pour chaque journée par l'API, valeurs réelles de Metz. La journée du 6 porte le lot de correction : remplacée, pas doublée. Relancez un lot déjà chargé : toujours 150 lignes, jamais une journée en double.

RÉFÉRENCE

Expressions et dépannage

Les expressions des deux pipelines

OÙ	EXPRESSION
Wait for batch, Expression	`@activity('Check batch').output.exists`, délai `0.00:10:00`
Check batch et Read batch, fichier	`batch_@{pipeline().parameters.batch_id}.json`
Route by mode, Expression	`@or(equals(activity('Read batch').output.firstRow.mode, 'standard'), equals(activity('Read batch').output.firstRow.mode, 'correction'))`
Load batch, paramètres	`@activity('Read batch').output.firstRow.batch_id`, `.mode`, `.file`, `.run_date`
Unknown mode, Message	`@concat('Mode de lot inconnu : ', activity('Read batch').output.firstRow.mode)`
Send report, Objet	`Lot @{pipeline().parameters.batch_id} chargé en mode @{activity('Read batch').output.firstRow.mode}`
Stage file, nom de fichier source	`@{pipeline().parameters.file}`
Get weather, URL relative	`/v1/forecast?latitude=49.12&longitude=6.18&daily=temperature_2m_max,precipitation_sum&timezone=Europe%2FParis&start_date=@{pipeline().parameters.run_date}&end_date=@{pipeline().parameters.run_date}`
Load day, temp_max et rain_mm	`@activity('Get weather').output.daily.temperature_2m_max[0]`, `@activity('Get weather').output.daily.precipitation_sum[0]`

COPIEZ DEPUIS EXPRESSIONS.TXT Un PDF coupe les lignes longues et ajoute un espace au copier-coller : `pipe line()` au lieu de `pipeline()` donne « Expression non reconnue ». Toutes les expressions de ce guide sont dans `expressions.txt`, fourni avec les données : copiez-les de là, une ligne entière à la fois.

DEUX FAÇONS D'ÉCRIRE Dans un champ texte (nom de fichier, URL relative, objet), l'expression s'insère entre `@{ }` au milieu du texte. Dans un champ expression (Until, Condition Si, Message, valeur d'un paramètre), elle commence par `@` et occupe tout le champ.

Si quelque chose bloque

SYMPTÔME	CAUSE ET CORRECTION
Le script SQL s'arrête sur `CREATE PROCEDURE`	Il doit être seul dans son lot : le `GO` avant lui est indispensable. Exécutez le script entier, pas une sélection.
`Stage file` échoue sur la mise en lots	La destination entrepôt exige le staging. Dans **Paramètres** du Copy, laissez **Activer la mise en lots** coché.
`Get weather` : 400 ou « out of allowed range »	La date demandée est trop ancienne pour l'API de prévision (trois mois) ou mal formée. Vérifiez `run_date` au format `AAAA-MM-JJ`.
`Load day` : « property 'daily' doesn't exist »	Le Web a échoué ou n'est pas relié devant la procédure. Reliez `Get weather` en vert vers `Load day`.
`Wait for batch` : « TimeLimitReached »	Le manifeste n'a jamais été trouvé. Les fichiers sont à la racine de Fichiers au lieu de `inbox`, ou le nom diffère d'une lettre. Pour vos essais, ramenez le délai à `0.00:01:00`.
Le pipeline réussit mais `fact_sales` reste vide	Comptez `stg_sales` et `fact_sales` (requête en fin de `expressions.txt`). Transit plein : `Load day` absent, non enregistré ou mal paramétré. Transit vide : `Stage file` n'a rien copié.
La condition part toujours en False	Casse ou espace dans l'expression ou dans le manifeste. Les modes sont `standard` et `correction`, en minuscules, entre apostrophes droites.
`Load batch` : paramètres absents	L'enfant n'a pas été enregistré après l'ajout de ses paramètres. Enregistrez `PL_Load_Day`, puis rouvrez l'activité.